Name : JETHAVA JAY VINODBHAI

Enrollment No. : 250283152004

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    auc,
    roc_curve,
    roc_auc_score,
    precision_recall_curve
)
```

Load Cancer Data

```
cancer = load_breast_cancer()
```

```
cancer
```

```
 'frame': None,
 'target_names': array(['malignant', 'benign'], dtype='<U9'),
 'DESCR': '.. _breast_cancer_dataset:\n\nBreast cancer wisconsin (diagnostic) dataset\n--------------------------------------------\n\n**Data Set
Characteristics:**\n\n:Number of Instances: 569\n\n:Number of Attributes: 30 numeric, predictive attributes and the class\n\n:Attribute Information:\n    - radius
(mean of distances from center to points on the perimeter)\n    - texture (standard deviation of gray-scale values)\n    - perimeter\n    - area\n    - smoothness
(local variation in radius lengths)\n    - compactness (perimeter^2 / area - 1.0)\n    - concavity (severity of concave portions of the contour)\n    - concave points
(number of concave portions of the contour)\n    - symmetry\n    - fractal dimension ("coastline approximation" - 1)\n\n    The mean, standard error, and "worst" or
largest (mean of the three\n    worst/largest values) of these features were computed for each image,\n    resulting in 30 features.  For instance, field 0 is Mean
Radius, field\n    10 is Radius SE, field 20 is Worst Radius.\n\n    - class:\n            - WDBC-Malignant\n            - WDBC-Benign\n\n:Summary
Statistics:\n\n===================================== ====== ======\n                                                Min    Max\n===================================== ======
======\nradius (mean):                        6.981  28.11\ntexture (mean):                       9.71   39.28\nperimeter (mean):                     43.79
188.5\narea (mean):                        143.5  2501.0\nsmoothness (mean):                    0.053  0.163\ncompactness (mean):                   0.019
0.345\nconcavity (mean):                  0.0    0.427\nconcave points (mean):                0.0    0.201\nsymmetry (mean):                      0.106
0.304\nfractal dimension (mean):          0.05   0.097\nradius (standard error):              0.112  2.873\ntexture (standard error):             0.36
4.885\nperimeter (standard error):        0.757  21.98\narea (standard error):                6.802  542.2\nsmoothness (standard error):          0.002
0.031\ncompactness (standard error):       0.002  0.135\nconcavity (standard error):           0.0    0.396\nconcave points (standard error):      0.0
0.053\nsymmetry (standard error):         0.008  0.079\nfractal dimension (standard error):   0.001  0.03\nradius (worst):                       7.93
36.04\ntexture (worst):                   12.02  49.54\nperimeter (worst):                    50.41  251.2\narea (worst):                         185.2
4254.0\nsmoothness (worst):               0.071  0.223\ncompactness (worst):                  0.027  1.058\nconcavity (worst):                    0.0
1.252\nconcave points (worst):            0.0    0.291\nsymmetry (worst):                     0.156  0.664\nfractal dimension (worst):            0.055
0.208\n===================================== ====== ======\n\n:Missing Attribute Values: None\n\n:Class Distribution: 212 - Malignant, 357 - Benign\n\n:Creator:  Dr.
William H. Wolberg, W. Nick Street, Olvi L. Mangasarian\n\n:Donor: Nick Street\n\n:Date: November, 1995\n\nThis is a copy of UCI ML Breast Cancer Wisconsin
(Diagnostic) datasets.\nhttps://goo.gl/U2Uwz2\n\nFeatures are computed from a digitized image of a fine needle\naspirate (FNA) of a breast mass.  They
describe\ncharacteristics of the cell nuclei present in the image.\n\nSeparating plane described above was obtained using\nMultisurface Method-Tree (MSM-T) [K. P.
Bennett, "Decision Tree\nConstruction Via Linear Programming." Proceedings of the 4th\nMidwest Artificial Intelligence and Cognitive Science Society,\npp. 97-101,
1992], a classification method which uses linear\nprogramming to construct a decision tree.  Relevant features\nwere selected using an exhaustive search in the space
of 1-4\nfeatures and 1-3 separating planes.\n\nThe actual linear program used to obtain the separating plane\nin the 3-dimensional space is that described in:\n[K. P.
Bennett and O. L. Mangasarian: "Robust Linear\nProgramming Discrimination of Two Linearly Inseparable Sets",\nOptimization Methods and Software 1, 1992, 23-
34].\n\nThis database is also available through the UW CS ftp server:\n\nftp ftp.cs.wisc.edu\ncd math-prog/cpo-dataset/machine-learn/WDBC/\n\n.. dropdown::
References\n\n  - W.N. Street, W.H. Wolberg and O.L. Mangasarian. Nuclear feature extraction\n    for breast tumor diagnosis. IS&T/SPIE 1993 International Symposium
on\n    Electronic Imaging: Science and Technology, volume 1905, pages 861-870,\n    San Jose, CA, 1993.\n  - O.L. Mangasarian, W.N. Street and W.H. Wolberg. Breast
cancer diagnosis and\n    prognosis via linear programming. Operations Research, 43(4), pages 570-577,\n    July-August 1995.\n  - W.H. Wolberg, W.N. Street, and O.L.
Mangasarian. Machine learning techniques\n    to diagnose breast cancer from fine-needle aspirates. Cancer Letters 77 (1994)\n    163-171.\n',
 'feature_names': array(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
        'mean smoothness', 'mean compactness', 'mean concavity',
        'mean concave points', 'mean symmetry', 'mean fractal dimension',
        'radius error', 'texture error', 'perimeter error', 'area error',
        'smoothness error', 'compactness error', 'concavity error',
        'concave points error', 'symmetry error',
        'fractal dimension error', 'worst radius', 'worst texture',
        'worst perimeter', 'worst area', 'worst smoothness',
        'worst compactness', 'worst concavity', 'worst concave points',
        'worst symmetry', 'worst fractal dimension'], dtype='<U23'),
 'filename': 'breast_cancer.csv',
 'data_module': 'sklearn.datasets.data'}
```

```
1 X = cancer.data
2 y = cancer.target
```

```
1 X
```

```
array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
        1.189e-01],
       [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
```

```
        8.902e-02],
       [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
        8.758e-02],
       ...,
       [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
        7.820e-02],
       [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
        1.240e-01],
       [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
        7.039e-02]])
```

```
1 X.shape
```

```
(569, 30)
```

```
1 y
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
       1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
       1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
       1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
       1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
       1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
       1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
       1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1])
```

```
1 y.shape
```

```
(569,)
```

```
1 df = pd.DataFrame(X,columns=cancer.feature_names)
2 df.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.60	2019.0	0.1622	0.6656
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158.80	1956.0	0.1238	0.1866
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152.50	1709.0	0.1444	0.4245
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	98.87	567.7	0.2098	0.8663
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	152.20	1575.0	0.1374	0.2050

5 rows × 30 columns

```
df["Target"] = y
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   mean radius              569 non-null    float64
 1   mean texture             569 non-null    float64
 2   mean perimeter           569 non-null    float64
 3   mean area                569 non-null    float64
 4   mean smoothness          569 non-null    float64
 5   mean compactness         569 non-null    float64
 6   mean concavity           569 non-null    float64
 7   mean concave points      569 non-null    float64
 8   mean symmetry            569 non-null    float64
 9   mean fractal dimension   569 non-null    float64
 10  radius error             569 non-null    float64
 11  texture error            569 non-null    float64
 12  perimeter error          569 non-null    float64
 13  area error               569 non-null    float64
 14  smoothness error         569 non-null    float64
 15  compactness error        569 non-null    float64
 16  concavity error          569 non-null    float64
 17  concave points error     569 non-null    float64
 18  symmetry error           569 non-null    float64
 19  fractal dimension error  569 non-null    float64
 20  worst radius             569 non-null    float64
 21  worst texture            569 non-null    float64
 22  worst perimeter          569 non-null    float64
 23  worst area               569 non-null    float64
 24  worst smoothness         569 non-null    float64
 25  worst compactness        569 non-null    float64
 26  worst concavity          569 non-null    float64
 27  worst concave points     569 non-null    float64
```

```
 28  worst symmetry           569 non-null    float64
 29  worst fractal dimension  569 non-null    float64
 30  Target                   569 non-null    int64
dtypes: float64(30), int64(1)
memory usage: 137.9 KB
```

```
1 df.describe()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	smo
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...	569.000000	569.000000	569.000000	569
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	0.062798	...	25.677223	107.261213	880.583128	0
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	0.007060	...	6.146258	33.602542	569.356993	0
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	0.049960	...	12.020000	50.410000	185.200000	0
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	0.057700	...	21.080000	84.110000	515.300000	0
50%	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	0.061540	...	25.410000	97.660000	686.500000	0
75%	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	0.066120	...	29.720000	125.400000	1084.000000	0
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	0.097440	...	49.540000	251.200000	4254.000000	0

8 rows × 31 columns

```
1 df["Target"].value_counts()
```

Target	count
1	357
0	212

dtype: int64

Split Data into Train And Test

```
1 X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42,stratify=y)
```

Apply Standard Scaling in Train and Test Data

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Apply Logistic Regression Model

```
model = LogisticRegression(max_iter=5000)
```

Train the model

```
model.fit(
    X_train,
    y_train
)
```

```
LogisticRegression
LogisticRegression(max_iter=5000)
```

Making Predictions on Test Data

```
y_pred = model.predict(X_test)
print(y_pred)
```

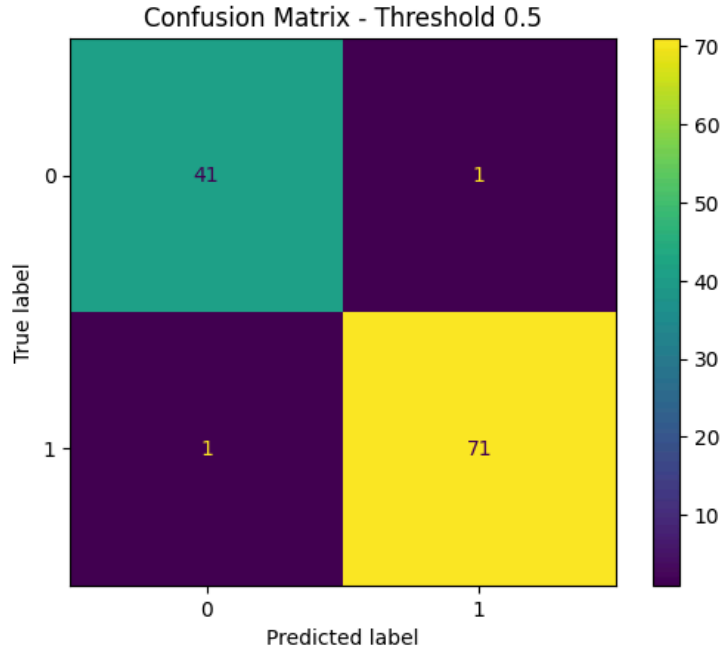
```
[0 1 0 1 0 1 1 0 0 0 1 0 1 0 0 1 0 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 1 0 0 1
 1 1 0 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 1 1 0
 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 1 1 1 0 0 0 1 0 1 0 1 0 0 0 1 0 0 1 0 1 0 1
 0 1 1]
```

Confusion Matrix - Threshold 0.5

```
cm = confusion_matrix(y_test,y_pred)
print(cm)
```

```
[[41  1]
 [ 1 71]]
```

```
display = ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=model.classes_)
display.plot()
plt.title("Confusion Matrix - Threshold 0.5")
plt.show()
```




```
accuracy = accuracy_score(y_test,y_pred)
precision = precision_score(y_test,y_pred)
recall = recall_score(y_test,y_pred)
f1 = f1_score(y_test,y_pred)

print("Accuracy (threshold = 0.5): ",accuracy)
print("Precision (threshold = 0.5): ",precision)
print("Recall (threshold = 0.5): ",recall)
print("F1 Score (threshold = 0.5): ",f1)
```

```
Accuracy (threshold = 0.5):  0.9824561403508771
Precision (threshold = 0.5):  0.9861111111111112
Recall (threshold = 0.5):  0.9861111111111112
F1 Score (threshold = 0.5):  0.9861111111111112
```

Getting Prediction Probabilities

```
y_prob = model.predict_proba(X_test)[:, 1]
print(y_prob[:10])
```

```
[5.88824186e-08 9.99988665e-01 6.41082462e-03 5.33508537e-01
 6.52500097e-10 9.92160398e-01 9.99982825e-01 5.63527494e-07
 5.43163692e-05 8.04184941e-11]
```

Prediction Using 0.3 Threshold

```
y_pred_03 = (y_prob >= 0.3).astype(int)
```

Model Evaluation at Threshold = 0.3

```
accuracy = accuracy_score(y_test,y_pred_03)
precision = precision_score(y_test,y_pred_03)
recall = recall_score(y_test,y_pred_03)
f1 = f1_score(y_test,y_pred_03)

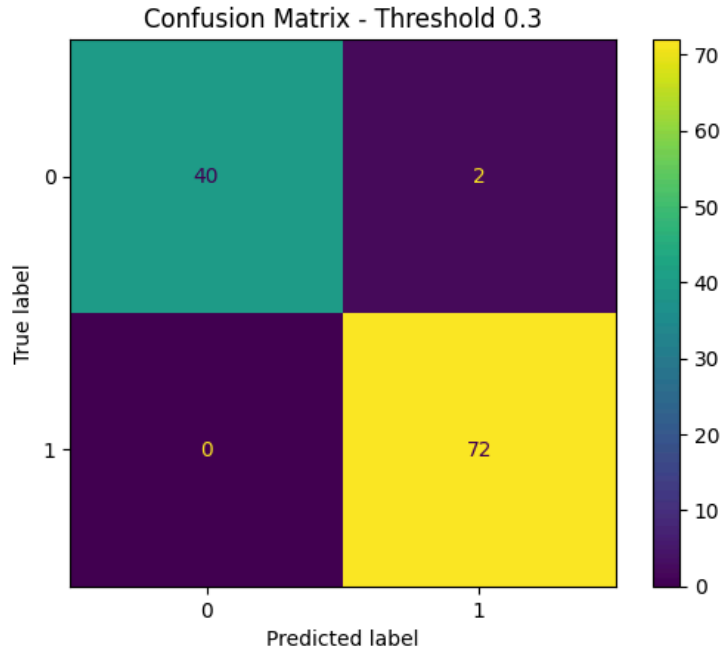
print("Accuracy (threshold = 0.3): ",accuracy)
print("Precision (threshold = 0.3): ",precision)
print("Recall (threshold = 0.3): ",recall)

print("F1 Score (threshold = 0.3): ",f1)
```

```
Accuracy (threshold = 0.3):  0.9824561403508771
Precision (threshold = 0.3):  0.972972972972973
Recall (threshold = 0.3):  1.0
F1 Score (threshold = 0.3):  0.9863013698630136
```

Confusion Matrix - Threshold 0.3

```
cm_03 = confusion_matrix(y_test, y_pred_03)
disp_03 = ConfusionMatrixDisplay(
    confusion_matrix=cm_03,
    display_labels=model.classes_
)
disp_03.plot()
plt.title("Confusion Matrix - Threshold 0.3")
plt.show()
```



Prediction Using 0.9 Threshold

```
y_pred_09 = (y_prob >= 0.9).astype(int)
```

Model Evaluation at Threshold = 0.3

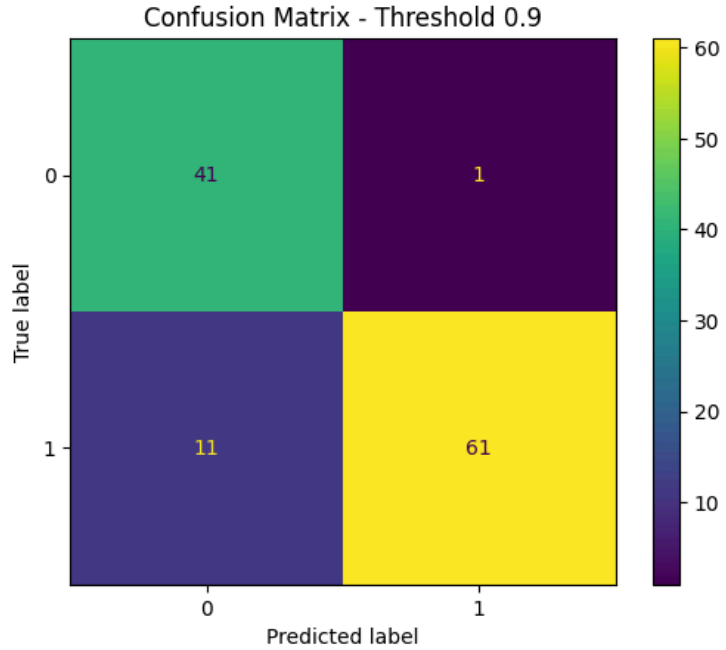
```
accuracy = accuracy_score(y_test,y_pred_09)
precision = precision_score(y_test,y_pred_09)
recall = recall_score(y_test,y_pred_09)
f1 = f1_score(y_test,y_pred_09)

print("Accuracy (threshold = 0.9): ",accuracy)
print("Precision (threshold = 0.9): ",precision)
print("Recall (threshold = 0.9): ",recall)
print("F1 Score (threshold = 0.9): ",f1)
```

```
Accuracy (threshold = 0.9):  0.8947368421052632
Precision (threshold = 0.9):  0.9838709677419355
Recall (threshold = 0.9):  0.8472222222222222
F1 Score (threshold = 0.9):  0.9104477611940298
```

Confusion Matrix - Threshold 0.9

```
cm_09 = confusion_matrix(y_test, y_pred_09)
disp_09 = ConfusionMatrixDisplay(
    confusion_matrix=cm_09,
    display_labels=model.classes_
)
disp_09.plot()
plt.title("Confusion Matrix - Threshold 0.9")
plt.show()
```



Receiver Operating Characteristic(ROC) Curve and Area Under the Curve(AUC)

```
fpr, tpr, roc_thresholds = roc_curve(y_test, y_prob)

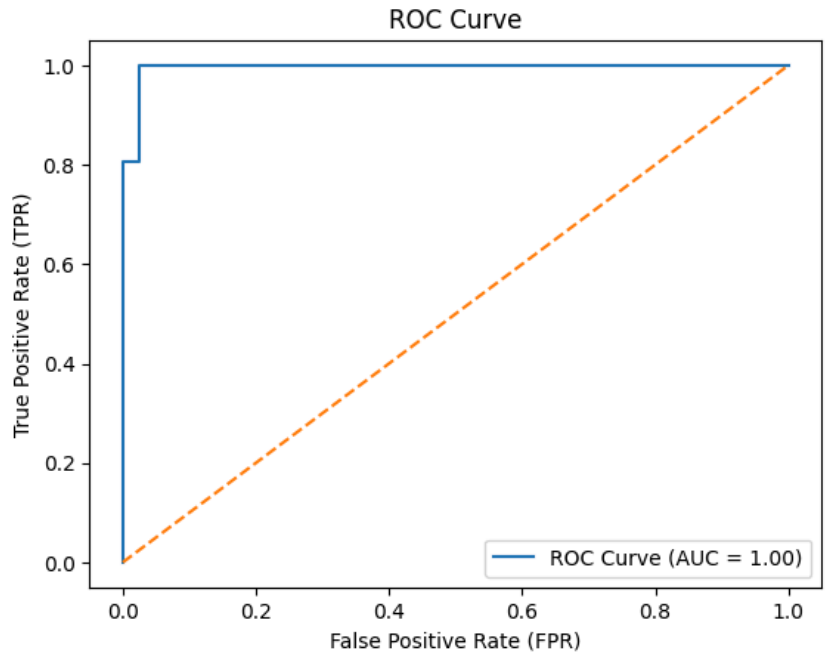
auc = roc_auc_score(y_test, y_prob)

plt.plot(fpr, tpr, label=f"ROC Curve (AUC = {auc:.2f})")
plt.plot([0, 1], [0, 1], linestyle="--")

plt.xlabel("False Positive Rate (FPR)")
plt.ylabel("True Positive Rate (TPR)")
```

```
plt.title("ROC Curve")
plt.legend()
plt.show()

print("AUC =", auc)
```




```
AUC = 0.9953703703703703
```

ROC Point for Threshold 0.5

```
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()

fpr_05 = fp / (fp + tn)
tpr_05 = tp / (tp + fn)

print("Threshold = 0.5")
print("False Positive Rate =", fpr_05)
print("True Positive Rate =", tpr_05)
```

```
Threshold = 0.5
False Positive Rate = 0.023809523809523808
True Positive Rate = 0.9861111111111112
```

ROC Point for Threshold 0.3

```
tn, fp, fn, tp = confusion_matrix(y_test, y_pred_03).ravel()

fpr_03 = fp / (fp + tn)
tpr_03 = tp / (tp + fn)

print("Threshold = 0.3")
print("False Positive Rate =", fpr_03)
print("True Positive Rate =", tpr_03)
```

```
Threshold = 0.3
False Positive Rate = 0.047619047619047616
True Positive Rate = 1.0
```

ROC Point for Threshold 0.9

```
tn, fp, fn, tp = confusion_matrix(y_test, y_pred_09).ravel()

fpr_09 = fp / (fp + tn)
tpr_09 = tp / (tp + fn)

print("Threshold = 0.9")
print("False Positive Rate =", fpr_09)
print("True Positive Rate =", tpr_09)
```

```
Threshold = 0.9
False Positive Rate = 0.023809523809523808
True Positive Rate = 0.8472222222222222
```

Display ROC Curve with All 3 Thresholds

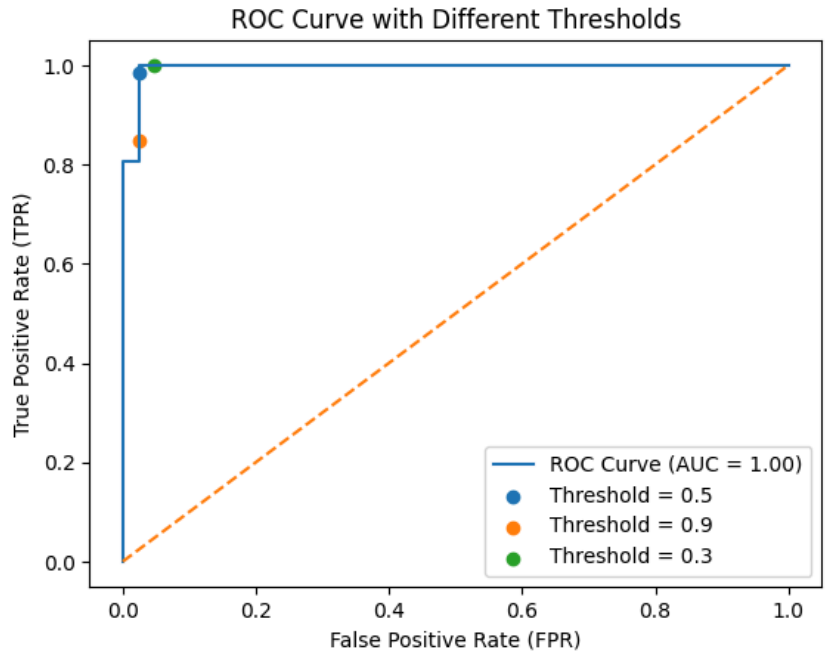
```
roc_auc_score_value = roc_auc_score(y_test, y_prob)
plt.plot(fpr, tpr, label=f"ROC Curve (AUC = {roc_auc_score_value:.2f})")

plt.scatter(fpr_05, tpr_05, label="Threshold = 0.5")
plt.scatter(fpr_09, tpr_09, label="Threshold = 0.9")
plt.scatter(fpr_03, tpr_03, label="Threshold = 0.3")

plt.plot([0, 1], [0, 1], linestyle="--")

plt.xlabel("False Positive Rate (FPR)")
plt.ylabel("True Positive Rate (TPR)")
plt.title("ROC Curve with Different Thresholds")
```

```
plt.legend()
plt.show()
```

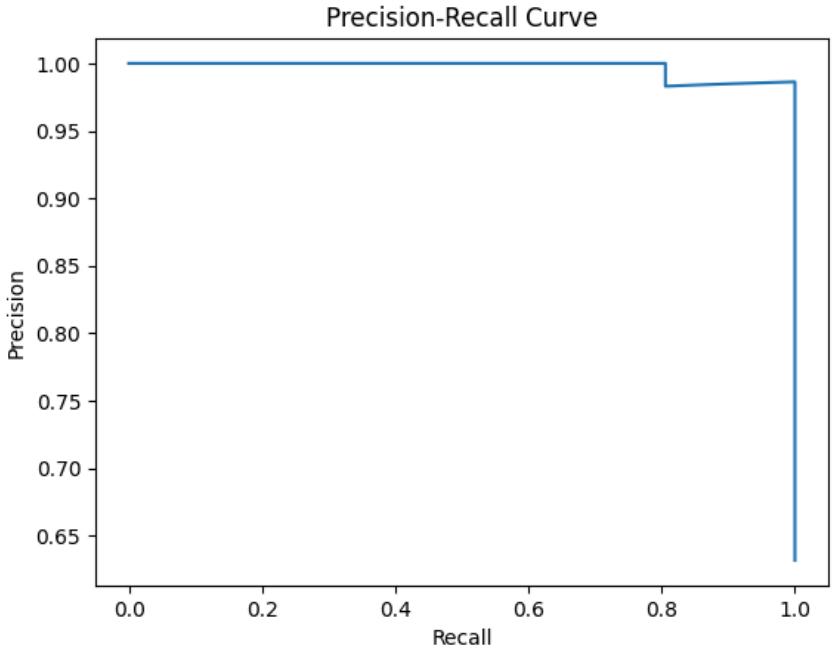


Precision-Recall Curve

```
precision, recall, pr_thresholds = precision_recall_curve(
    y_test, y_prob
)

plt.plot(recall, precision)

plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.show()
```



Precision & Recall at Threshold 0.5

```
precision_05 = precision_score(y_test, y_pred)
recall_05 = recall_score(y_test, y_pred)

print("Threshold = 0.5")
print("Precision =", precision_05)
print("Recall =", recall_05)
```

```
Threshold = 0.5
Precision = 0.9861111111111112
Recall = 0.9861111111111112
```

Precision & Recall at Threshold 0.3

```
precision_03 = precision_score(y_test, y_pred_03)
recall_03 = recall_score(y_test, y_pred_03)

print("Threshold = 0.3")
print("Precision =", precision_03)
print("Recall =", recall_03)
```

```
Threshold = 0.3
Precision = 0.972972972972973
Recall = 1.0
```

Precision & Recall at Threshold 0.9

```
precision_09 = precision_score(y_test, y_pred_09)
recall_09 = recall_score(y_test, y_pred_09)

print("Threshold = 0.9")
print("Precision =", precision_09)
print("Recall =", recall_09)
```

```
Threshold = 0.9
Precision = 0.9838709677419355
Recall = 0.8472222222222222
```

Precision-Recall Curve + 3 Threshold Dots

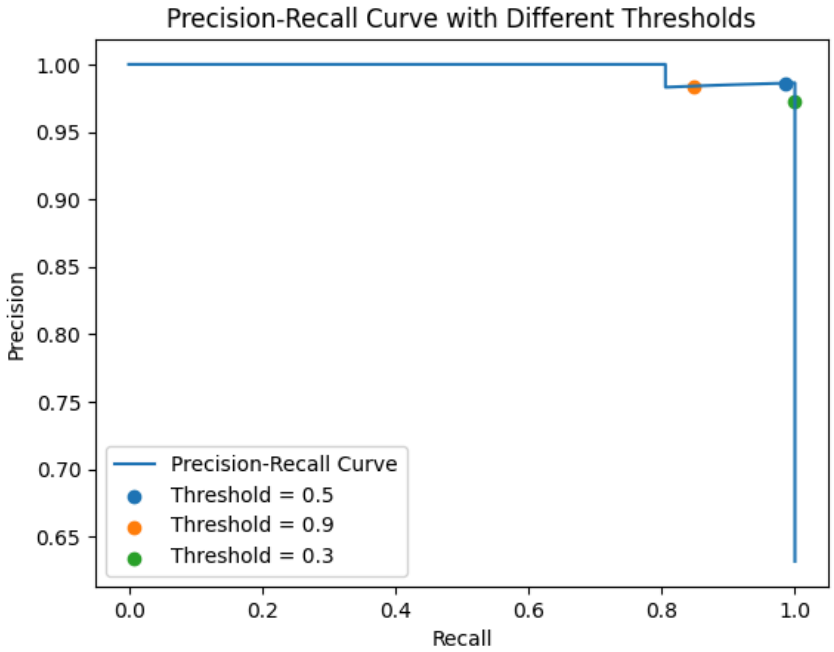
```
plt.plot(recall, precision, label="Precision-Recall Curve")

# Threshold = 0.5
plt.scatter(
    recall_05,
    precision_05,
    label="Threshold = 0.5"
)

# Threshold = 0.9
plt.scatter(
    recall_09,
    precision_09,
    label="Threshold = 0.9"
)

# Threshold = 0.3
plt.scatter(
    recall_03,
    precision_03,
    label="Threshold = 0.3"
)

plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve with Different Thresholds")
plt.legend()
plt.show()
```


Precision-Recall Curve with Different Thresholds
Precision
Recall
1.00
0.95
0.90
0.85
0.80
0.75
0.70
0.65
0.0
0.2
0.4
0.6
0.8
1.0
Precision-Recall Curve
Threshold = 0.5
Threshold = 0.9
Threshold = 0.3

```
1 df
```